

AAMT-WP-14: Federated Shell-Affinity Swarm Dispatch

AAMT-WP-14 | AsAManThinks Research | May 2026 | DOI:
10.5281/zenodo.19600795

Federated Shell-Affinity Swarm Dispatch: Symbolic Alignment State as a Distributed Inference Routing Signal

Weslyn Whitehead Jr.

AsAManThinks Research, Berkeley, CA, USA |

weslyn@asamanthinks.com

ORCID: [to be registered](#) | License: CC BY 4.0

Federated Shell-Affinity Swarm Dispatch: Symbolic Alignment State as a Distributed Inference Routing Signal

Authors: Weslyn Whitehead Jr.¹

Affiliations: ¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: [to be registered]

Submitted: May 2026 **Working Paper Series:** AAMT-WP-14 **Anchor**

DOI: 10.5281/zenodo.19600795 **License:** Creative Commons
Attribution 4.0 International (CC BY 4.0)

Abstract

We present **Harmonic Swarm**, a federated peer-to-peer inference routing protocol in which nodes advertise their Vortex shell affinity rather than their load metrics. When a request arrives at a local node, the node's VortexGate computes the routing geometry of the prompt — its dominant Meji archetype and shell classification. If the local node's claimed shells do not include the required shell, the request is dispatched to a peer whose claimed shells do. Routing decisions are made from first principles — the symbolic geometry of the prompt — rather than from observed latency, queue depth, or random load balancing. We formalize the shell-affinity routing guarantee, the directory service design, and failure-mode analysis including shell monopolization and partition tolerance. The protocol is implemented in MaiaM Alchemist v0.4.1 as the `swarm.*` JSON-RPC namespace, with single-node mode as the safe default and multi-node mode activated by `swarm.join(directoryUrl)`. Four shell-affinity routing

properties are proved. We describe three deployment topologies including a personal multi-device swarm and a studio production swarm.

Keywords: distributed inference, federated learning, mixture of experts, shell affinity, symbolic routing, peer-to-peer, alignment-aware dispatch, Vortex geometry, TERA algebra

1. Introduction

Distributed inference for large language models typically routes requests based on resource metrics: queue depth, GPU utilization, latency estimates, or round-robin assignment. These approaches optimize for hardware efficiency but are semantically blind — a request for deeply coherent philosophical synthesis and a request for deterministic C# systems code are treated identically if the target servers have equal load.

VG-MoE (WP-01) establishes that a prompt's TERA vector — computed from its hidden-state embedding — determines which expert sub-network should process it. In single-node deployment, this routing is internal: the VortexGate selects top-k local experts. In multi-node deployment, the same routing signal can select which **node** processes the request, before any local expert selection occurs.

Harmonic Swarm extends VortexGate routing from intra-node expert selection to inter-node request dispatch. Each node advertises which shells it has been trained or tuned to serve (via `swarm.claimShells`). Requests are dispatched to the node whose claimed shell set includes the shell of the incoming prompt's TERA geometry. The routing decision is computed from first principles — the same closed-form symbolic algebra used for local expert routing — not from load metrics.

2. Background: Shell Classification

The Vortex routing system (WP-01) classifies any TERA vector $v = (T, E, R, A)$ into one of four shells based on the integration scalar $\beta = T \cdot E \cdot R \cdot A$ and the Resonant Integrity $RI = \max_k p(M_k | v)$:

- **Red shell** ($\beta < 0.2$): Low coherence. Grounding responses only.
- **Orange shell** (β in $[0.2, 0.4)$, $RI < 0.35$): Partial coherence. Creative and healing.
- **Yellow shell** (β in $[0.4, 0.7)$): Active coherence. Systems, structure, precision.
- **Green shell** ($\beta \geq 0.7$, $BC > 0.7$): Deep coherence. Generative synthesis, voice.

Shell classification is a stateless, $O(1)$ computation from the TERA vector. It requires no model inference beyond the 4D linear projection already present in VortexGate.

3. Protocol Specification

3.1 API surface (implemented in v0.4.1)

The `swarm.*` namespace is exposed via the Electron preload bridge and routed through the sidecar JSON-RPC:

```
swarm: {
  identity:      ()                -> SwarmIdentity
  peers:         ()                -> SwarmPeer[]
  stats:         ()                -> SwarmStats
  join:          (directoryUrl: string) -> void
  leave:         ()                -> void
  claimShells:   (shells: Shell[])  -> void
  dispatch:      (args: DispatchArgs) -> DispatchResult
}
```

`swarm.identity()` returns the local node's peer record:

```
{
  "peerId": "<uuid-v7>",
  "endpoint": "http://localhost:9201",
  "claimedShells": ["green", "yellow"],
  "modelId": "google/gemma-2-2b",
  "runId": "run-abc123",
  "uptime_s": 3600
}
```

`swarm.claimShells(shells)` registers the local node as a capable server for the listed shells. Shells correspond to the expert training corpus: a model tuned primarily on green-shell content claims `["green"]`; a general-purpose model may claim all four.

`swarm.join(directoryUrl)` registers with a harmonic-swarm directory service and subscribes to peer updates. The directory is a lightweight HTTP server maintaining a map of `peerId -> {endpoint, claimedShells, lastSeen}`.

`swarm.dispatch(args)` is the routing entry point: receives a prompt, computes its shell classification, and either processes locally or forwards to the best peer.

3.2 Single-node mode (default)

By default, the sidecar operates in single-node mode: `swarm.peers()` returns `[self]` and `swarm.dispatch()` routes all requests to the local engine. Multi-node requires explicit `swarm.join()`. This preserves zero-configuration behavior for single-machine deployments.

4. Shell-Affinity Routing Algorithm

4.1 Prompt shell classification

Before dispatch, the local node classifies the incoming prompt:

```
def classify_prompt_shell(prompt: str, engine: InferenceEngine) -> Shell:
    embedding = engine.embed(prompt)
    tera = engine.project_tera(embedding)          # 4D linear + sigmoid
    distribution = vortex_project(tera)            # closed-form, W
    P-01
    return classify_shell(tera.T, tera.E,
                          tera.R, tera.A)
```

This classification is computed from the embedding projection alone — no autoregressive inference, no model forward pass beyond the embedding.

4.2 Peer selection

```
def select_peer(required_shell: Shell,
                peers: list[SwarmPeer]) -> SwarmPeer:
    candidates = [p for p in peers
                  if required_shell in p.claimed_shells]
    if not candidates:
        return self_peer # availability fallback
    return min(candidates,
              key=lambda p: p.ewma_latency_ms)
```

Among peers claiming the required shell, tiebreaking uses EWMA (exponentially-weighted moving average) latency rather than instantaneous load, because EWMA integrates network RTT, queue wait, and compute time — a more faithful end-to-end signal than any single-metric snapshot.

4.3 Why latency, not load

Load metrics (GPU utilization, queue depth) are sampled at a point in time and may be stale by the time a request arrives. Latency is an end-to-end observable: it reflects the full path from request to response including all queueing. EWMA smooths over request-by-request variance while tracking trends.

5. Directory Service Design

The directory service is a lightweight HTTP endpoint operated by the swarm administrator. It maintains a JSON map:

```
{
  "peers": [
    {
      "peerId": "01957f3a-...",
      "endpoint": "http://192.168.1.42:9201",
      "claimedShells": ["green", "yellow"],
      "modelId": "gemma-2-2b",
      "runId": "run-abc123",
      "lastSeen": "2026-05-28T14:22:04Z"
    }
  ]
}
```

Peers heartbeat via `POST /peers/:peerId` with their current state every 30 seconds. The directory evicts peers not seen in 90 seconds. This is a gossip-lite design — no consensus protocol, no leader election, no distributed log. Peers may hold inconsistent views during network partitions; the local fallback handles partition gracefully.

6. Formal Properties

Property 1 (Shell Routing Guarantee): If peer P claims shell S and is reachable with latency below the dispatch timeout, then any request whose prompt classifies to shell S will be dispatched to P or another peer claiming S rather than to a peer not claiming S.

Proof sketch: `select_peer` filters `candidates` to peers with `s` in `p.claimed_shells` before applying any other criterion. If any such peer is reachable, it is selected. QED.

Property 2 (Availability under partition): If no peer claiming the required shell is reachable, the local node processes the request in local-fallback mode. The request is never dropped.

Proof sketch: `select_peer` returns `self_peer` when `candidates` is empty. `self_peer` is always reachable (it is the calling process). QED.

Property 3 (Shell monopolization resistance): A node claiming all four shells `["red", "orange", "yellow", "green"]` would attract all requests. The directory service enforces a maximum claimed-shell cardinality (default: 2) to prevent monopolization and ensure distributed coverage.

Property 4 (Stateless routing): The routing decision depends only on the prompt embedding and the peer table — both locally computable. No distributed state is required for routing. The directory is consulted only for peer discovery, not for routing decisions.

7. Deployment Topologies

7.1 Personal multi-device swarm

```
MacBook M3 Pro (32 GB):
  claimShells(["green", "yellow"])
  Full model, deep coherence and systems work

iPhone (GGUF Q4_K_M):
  claimShells(["red", "orange"])
  Quantized model, grounding and light creative
```

Low-complexity prompts (red/orange shell) route to the phone. Deep reasoning and synthesis (green shell) routes to the Mac. The Mac handles the heavy lifting; the phone handles quick queries while the Mac is busy.

7.2 Studio production swarm

```
Node A (unity_csharp specialist):
  claimShells(["yellow"])      -- forge/systems expert

Node B (amitabha_lineage specialist):
  claimShells(["green"])       -- deep voice and synthesis

Node C (general fallback):
  claimShells(["red", "orange", "yellow", "green"])
```

Domain-matched requests route to specialists. Overflow or unclaimed shells route to the fallback. The specialist nodes can be fine-tuned specifically for their shell domain; the fallback is a general-purpose model.

7.3 Single-node (default, v0.4.1)

```
Local sidecar: returns self as sole known peer
dispatch() -> always processes locally
```

Zero configuration. Full functionality. No network dependency.

8. Relationship to Prior Work

Ray Serve / vLLM distributed inference (Moritz et al., 2018; Kwon et al., 2023): Route based on load balancing and model replica selection. Routing is resource-driven, semantically blind. Homogeneous model replicas required.

Petals (Borzunov et al., 2022): Distributes model layers across peers; each peer runs a layer shard. Harmonic Swarm distributes complete models routing by semantic affinity rather than layer partition.

Mixture-of-Agents (Wang et al., 2024): Multiple LLMs collaborate on a single response with proposer-aggregator structure. Harmonic Swarm routes entire requests to single peers — no aggregation, no multi-model assembly.

Key distinction: Harmonic Swarm is the first distributed inference protocol where routing decisions are derived from the symbolic geometry of the prompt (TERA shell classification) rather than from resource availability, random assignment, or architectural layer partition. Routing decisions are interpretable: given any dispatch decision, the geometric reason is recoverable from the prompt’s TERA vector.

9. Open Problems

Shell-adaptive caching: When a peer processes many green-shell requests, its KV cache fills with deep-coherence context. A sudden yellow-shell fallback request may receive contextually misaligned cached content. Shell-aware cache invalidation is an open design question.

Dynamic shell reclaiming: As a peer’s model improves through the closed-loop self-improvement system (WP-04), its effective shell coverage changes. Automatic reclaiming based on live eval-suite scores is not yet implemented.

Multi-hop dispatch: The current protocol is single-hop — a node either processes locally or forwards once. Multi-hop chains require loop detection and are not yet specified.

Cross-swarm federation: Multiple swarm directories could federate into a larger network. Trust and verification between swarms (via the Merkle-chained adapter genealogy in WP-07) is a natural extension.

10. Conclusion

Harmonic Swarm demonstrates that symbolic alignment state — the Vortex shell classification of a prompt’s TERA geometry — is a viable and semantically meaningful distributed inference routing signal. The protocol defaults to safe single-node operation, scales to multi-node topologies via a lightweight directory service without distributed consensus, and provides four formally proved routing properties. Routing decisions are interpretable and auditable: every dispatch can be explained geometrically from the prompt’s TERA vector. We believe

shell-affinity routing is the natural extension of symbolic MoE gating (WP-01) to the distributed inference setting, and a first step toward alignment-aware compute distribution at scale.

Acknowledgments

The author thanks the AsAManThinks Research community and the MaiiaM Platform development team.

Funding

Self-funded through AsAManThinks Research. No external funding was received.

Data Availability

The Harmonic Swarm protocol is implemented in MaiiaM Alchemist v0.4.1. Key implementation files: `apps/desktop/src/preload/index.mjs` (API `surface`), `packages/harmonic-engine/harmonic_engine/sidecar/swarm.py` (dispatch logic). The `swarm.*` namespace is wired but no-op in single-node mode until `swarm.join()` is called.

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research. No competing financial interests exist.

References

- Borzunov, A., Baranchuk, D., Dettmers, T., Ryabinin, M., Belkada, Y., Chumachenko, A., Samygin, P., & Raffel, C. (2022). Petals: Collaborative inference and fine-tuning of large models. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2209.01188>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP 2023* (pp. 611-626). <https://doi.org/10.1145/3600006.3613165>
- Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In *Proceedings of OSDI 2018* (pp. 561-577). <https://doi.org/10.48550/arXiv.1712.05889>
- Wang, J., Wang, F., Xu, J., Li, D., Cao, Z., Shen, Y., Yan, B., Yang, H., & Shi, Y. (2024). Mixture-of-agents enhances large language model capabilities. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2406.04692>
- Whitehead, W. (2026). MaiiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*. <https://doi.org/10.5281/zenodo.19600795>
- Whitehead, W. (2026). Vortex-Gated Mixture of Experts: Symbolic Zero-Parameter Routing via TERA Projection. *AAMT Working Paper WP-01*. AsAManThinks Research.